



Your Keys. Your Data. Your Choice of Cloud.

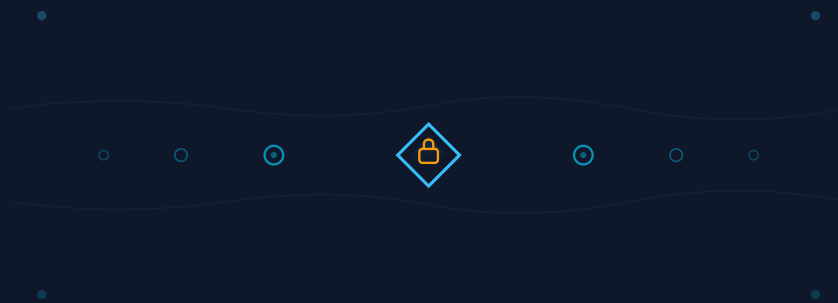
Customer-Managed Keys and Encrypted Data That Survives a Vendor Switch

Filesystem-native encryption on btrfs snapshots

[Rediacc](#) | 2026

Who Can Decrypt Your Backups Today?

The Adversary Model Most Backup Architectures Quietly Accept



The interesting question for a backup encryption review is not "is AES-256 enabled." It is: who holds the key, who can be subpoenaed for it, who can be coerced, and what survives a compromise of the vendor's own systems.

The **U.S. CLOUD Act** (18 U.S.C. §2713) compels any U.S.-based provider to produce data stored anywhere in the world on lawful request. If your backup vendor or their KMS provider holds the encryption key, that key is reachable through the vendor, not through you.

The legal process never touches your tenancy. You may not be notified.

The same shape recurs for vendor compromise. A breach of the backup vendor's control plane that exposes the KEK hierarchy exposes every customer whose ciphertext is bound to vendor-held keys. **Meta's €91M GDPR fine (Sept 2024) for plaintext-stored passwords is the analogue at the backup layer: vendor-accessible decryption material alongside customer ciphertext, same auditor verdict, larger blast radius.**

Encryption sovereignty is not "we support BYOK." It is "the key required to read your data is not in our possession, not in our supply chain, and not in the legal reach of a jurisdiction we cannot influence." Everything else is access control.

Schrems II Is an Architecture Requirement, Not a Policy

Why "Customer-Managed Keys" Stopped Being a Best Practice



Since the [CJEU Schrems II decision](#) (July 2020), encryption is not just a security measure under GDPR Article 32. The [EDPB Recommendations 01/2020](#) lists encryption as the primary supplementary technical measure for transfers out of the EU, with one binding condition: the data exporter alone controls the encryption keys, and those keys cannot be accessed by authorities in the recipient country. Vendor-managed keys held by a U.S. provider fail that test by construction.

Article 32 violations carry [€10 million or 2% of global turnover](#); broader Article 34 / consent / transfer violations carry [€20 million or 4%](#). Cumulative GDPR fines reached approximately [€5.88 billion](#) by January 2025. The Meta €91 million plaintext-password fine is the cleanest precedent that storage-side cryptographic posture is a fineable line item, not a policy choice.

[HIPAA's encryption safe harbor](#) at 45 CFR §164.402 sits in the same shape: if PHI is encrypted to HHS standards and the decryption key has not been compromised, the incident is excluded from breach notification entirely. Average healthcare breach cost exceeds [\\$10 million](#) (IBM Cost of a Data Breach 2024), so the safe harbor is the single highest-value control a covered entity owns. Whether it applies depends on a fact the auditor will examine: did the vendor have access to the key when the incident occurred.

The audit question to take into your next review: produce the cryptographic-key inventory and identify, for each backup dataset, the single legal entity that can compel decryption. If the answer is anything other than your own organization, your Schrems II and HIPAA safe-harbor posture is policy, not architecture.

Penalty Grid by Regulation

What Each Framework Costs If Encryption Posture Fails

Regulation	Encryption Mandate	Maximum Exposure	Notable Enforcement
GDPR Art. 32 / 34	Effectively required for cross-border post-Schrems II	€20M or 4% global turnover	Meta €91M (Sept 2024, plaintext passwords); cumulative ~€5.88B by Jan 2025
HIPAA	Addressable; safe harbor if keys uncompromised	\$68,928/violation; \$2.07M/year cap	Advocate Health \$5.55M settlement (unencrypted laptops); \$3M settlement citing mobile encryption failure
PCI-DSS 4.0	Required across all networks, expanded scope April 1, 2025	\$5,000-\$100,000/month from card brands	Disk encryption alone no longer sufficient for online systems; documented cryptographic architecture required
CCPA / CPRA	Liability tied to encryption status	\$100-\$750/consumer/incident statutory damages	100,000 - consumer breach = \$10-75M statutory exposure if unencrypted
SEC Cybersecurity	Material-incident disclosure within 4 business days	Up to \$25M plus trading consequences	Form 8-K required Dec 2023; encryption status is a materiality input
NIST 800-171	FIPS-validated cryptography required	Federal contract loss + False Claims Act exposure	Applies to all CUI handling

The cross-framework observation: no single encryption configuration satisfies every regulation, but customer-managed keys with full audit trails are the smallest configuration that satisfies all of them simultaneously. Two facts decide whether a control passes audit: who held the key at the moment of compromise, and whether you can prove it.

The decision an architect actually owns: which audits you want to be answering separately, and which you want to collapse into one cryptographic-architecture document.

The Triple Tax on Encrypted Data You Want to Move

Egress + Decrypt Compute + Re-encrypt Compute, Per Migration

When ciphertext is bound to a single vendor's key hierarchy or a single cloud's KMS, switching providers means decrypt at source, transfer plaintext (or re-encrypted), re-encrypt at destination. The bill has three lines.

Egress alone runs **\$0.08-\$0.15 per GB** across major clouds. 1 TB costs **\$90-\$300** to move; 1 PB costs **\$90,000-\$300,000** before any compute.

Apple reportedly pays **\$50 million per year** in data egress charges to AWS. The anchor matters: this is what the bill looks like at scale before any decrypt-re-encrypt compute is added on top.

At 10 Gbps sustained, 1 PB takes **9-10 days** of continuous transfer before re-encryption begins. End-to-end migrations run **3-6 months** for standard estates, **12+ months** for complex ones.

The Thales 2025 Cloud Security Study (S&P Global 451 Research, 3,200 respondents, 20 countries) measured the fragmentation directly: **57% of organizations use five or more key management systems** (up from 53%), **48% still manage keys via cloud provider consoles**, average enterprise runs **2.1 public cloud providers** and **85 SaaS applications**. Each new vendor relationship adds another bound ciphertext region.

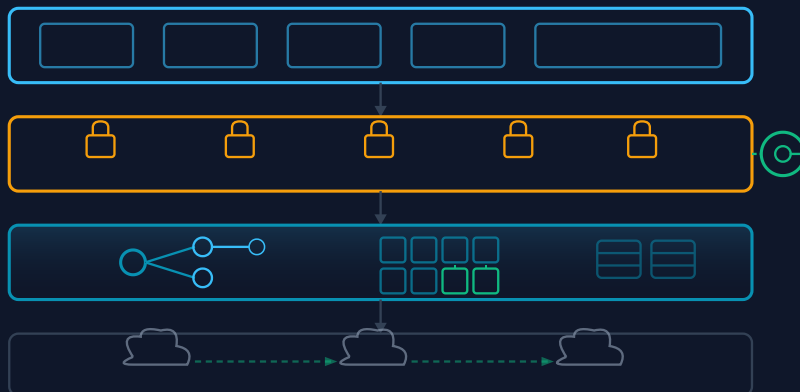
Downtime dominates the bill. Gartner's 2024 downtime cost survey puts the figure above **\$300,000/hour** for businesses without functional backup.

Gartner forecasts that by 2027, **more than 60% of organizations will adopt centralized multicloud KMaaS**, explicit recognition that per-provider encryption is no longer tenable.

The architectural question worth taking into your next migration estimate: how much of the cost is data transfer, and how much is decrypt-and-re-encrypt that exists only because ciphertext is bound to a vendor.

How btrfs Removes the Decrypt-and-Re-encrypt Tax

Filesystem-Native Encryption on Copy-on-Write Snapshots



btrfs stores all data and metadata in B-trees. A write creates a new physical block and updates the pointer; the original block is untouched.

A snapshot is a pure metadata operation: a new B-tree root pointing at the same blocks. No data is copied. Read references existing blocks, writes create new blocks, the filesystem maintains both states through pointer management.

Encryption is applied at the filesystem layer, below the application and above the storage hardware. Every block written to disk is encrypted under a customer-managed key before it reaches persistent storage. Because snapshots reference blocks by pointer and blocks are encrypted in place, a snapshot of encrypted data is encrypted by construction; no separate snapshot-encryption pass exists or is needed.

Mechanism: Rdiacc uses LUKS block-device encryption beneath the btrfs filesystem, with an AES-256-GCM cipher and a customer-generated 4096-bit RSA keypair as the envelope (`rdc keygen production`). On the wire, btrfs send/receive carries the already-encrypted blocks. A customer with the key can mount a recovered volume on any Linux system with `cryptsetup` and `btrfs-progs` using `cryptsetup open <device> rdiacc-vol && mount /dev/mapper/rdiacc-vol /mnt/recover` . No Rdiacc software is required for the read path.

btrfs send/receive transmits the delta between two snapshots as a stream of already-encrypted blocks. The receiving filesystem stores the encrypted blocks and continues. No decrypt step at the source, no re-encrypt step at the destination, no plaintext on the wire.

Migration becomes a data transfer; encryption is not part of the work. The \$90K-\$300K/PB egress remains (someone still has to move the bytes), but the multi-month decrypt-and-re-encrypt compute step disappears entirely.

The portability claim has a stronger form than "we support BYOK." Encrypted btrfs snapshots are standard filesystem objects, and a customer holding the encryption key can mount and read their backup on any Linux system with btrfs using stock tools, without Rdiacc software involved. That property is the consequence of building on an open

Key Lifecycle Under the Customer

Generation, Distribution, Rotation, Audit, All Customer-Side



Generation. Customers create encryption keys on their own infrastructure or via their preferred KMS; Rediacc does not generate, store, or transit customer keys. The encryption agent runs on the customer's backup endpoint, not on Rediacc-operated infrastructure. This is the property that makes the CLOUD Act and vendor-breach threat models tractable: no subpoena, CLOUD Act request, or control-plane compromise can produce key material Rediacc does not possess.

Distribution. Keys are presented to backup endpoints only at the points of encryption and decryption, through customer-controlled channels, and are not cached, stored, or relayed through Rediacc infrastructure. This is what the EDPB Recommendations 01/2020 require of an effective Schrems II supplementary measure: the exporter, and only the exporter, holds the keys.

Rotation. Customer-initiated rotation does not require re-encryption of existing snapshots: old snapshots remain decryptable with the old key, new snapshots use the new key, and the keyring evolves over time. This eliminates the operational stall behind the rotation-implementation gap most enterprises still report.

Audit. Every encryption operation is logged with immutable timestamps recording who accessed what, when, from where, answering GDPR Article 32, HIPAA safe-harbor, PCI-DSS 4.0, and SOC 2 CC6.1 evidence requests from one line. Consolidating backup-layer encryption to one interface removes the most architecturally consequential of the five-plus audit pipelines that **57% of organizations** currently run.

Audit prompt: when you ask your team for the current encryption-key inventory, how many systems do they need to query to produce it.

Where Other Backup Vendors Hold the Key

Mechanism, Not Marketing Posture



Veeam uses AES-256 with password-derived keys stored in the Veeam configuration database; BYOK via Azure Key Vault is available only for Veeam Data Cloud for Microsoft 365 Premium (limited preview, 2024) on top of KMIP 1.4 support. Recovery requires the original encryption password or access to the configuration database. The encryption is real; the dependency is on Veeam's configuration store, not solely on the key.

Rubrik runs a four-level hierarchy (DEK, KEK, Root Key, Customer Master Key) with BYOK via AWS KMS, Azure Key Vault, and external KMIP. The constraint is that backup data lives in Rubrik's proprietary Atlas filesystem and cannot be read by anything other than Rubrik, regardless of who holds the keys. Key sovereignty without data-format portability is half the problem solved.

Druva is SaaS-only on AWS with BYOK via AWS KMS alone (no Azure or GCP key sources), and BYOK keys operate inside Druva's envelope-encryption scheme so a customer cannot independently decrypt an exported blob even with their own key in hand. The combination of proprietary format, single cloud, and single KMS source produces the strongest vendor binding in the category.

Commvault centralizes keys in the CommServe database with optional KMIP integration (Thales CipherTrust, SafeNet). A "Media Password" option stores keys on backup media for offline recovery, a partial portability escape hatch. Without CommServe plus credentials, encrypted backup media is not recoverable.

Cohesity uses proprietary SpanFS with internal or external KMIP KMS. Two architectural ratchets: once encryption is enabled it cannot be disabled, and once external KMS is configured the system cannot revert to internal KMS. Strong forward posture; equally strong commitment.

Worth quantifying: for each of these vendors in your environment, identify the single component whose loss makes the encrypted data unrecoverable. That component, not the key, is your real dependency.

Cloud KMS Binds Ciphertext to a Provider by Design

And What That Means for Your Exit Plan

The three hyperscaler KMS offerings are not accidentally non-portable. The binding is architectural.

AWS KMS embeds the key ID into the ciphertext header, so data encrypted with an AWS KMS key can only be decrypted by that same key, in that same account, in that same region, and the keys themselves are not exportable. This is documented behavior, not a limitation to be patched.

Azure Key Vault supports key backup and restore, but a backed-up key can only be restored to another Azure Key Vault. Ciphertext encrypted under an Azure key is bound to the Azure ecosystem in the same way.

Google Cloud KMS ciphertext is GCP-specific. The one structural escape valve in the cloud-KMS landscape is **Google's External Key Manager (EKM)**, which keeps the key material in an external HSM the customer controls; Google holds the wrap-and-unwrap channel, not the key itself. EKM exists precisely because the default binding is otherwise complete.

Dedicated KMS platforms (Thales CipherTrust, HashiCorp Vault, Fortanix) are designed for cross-cloud portability. CipherTrust offers single-pane management across major clouds with FIPS 140-3 Level 3.

HashiCorp Vault provides open-source key management with multi-cloud distribution, and Fortanix differentiates on confidential computing via Intel SGX enclaves. These are the architecturally correct solutions if backup-layer portability is not part of the design.

The architectural choice underneath the buying decision: solve key portability above the backup layer (dedicated KMS) and accept proprietary backup formats, or solve both at the backup layer (filesystem-native encryption on an open format) and accept fewer enterprise-feature checkmarks elsewhere.

Next thing to think about: for each cloud KMS in production, identify the smallest unit of data that, if you needed to move it out today, you could not move without re-encryption.

Your Keys. Your Data. Your Choice of Cloud.

Customer-managed keys, filesystem-native encryption, and backup data that remains readable on any Linux system with btrfs.

The audit question worth answering: who currently holds the key required to decrypt your backups, and what jurisdiction can compel them.

rediacc.com/encryption-control

rediacc.com